Solving the Travelling Salesman Problem

Design and Development Report

Riyyan Ahmed Salaheddine Moustafid

M1 AI, Université Grenoble Alpes
Mid-term checkpoint, 18 October 2026

Repository: `https://github.com/riyyan-ahmed/TSP-Solver-Project` (branch `riyyan-feature`)

Abstract

This report describes the development of a C++ solver for the metric Travelling Salesman Problem. We cover the chronology of the work, the tools we relied on, the structure of the code and the reasons behind the main design choices. At the time of writing, the program reads both TSPLIB formats required by the specification, runs the trivial method and writes tours in the required format. A test suite of 92 cases covers the parser, the graph classes, the tour writer and the compiled executable. The tours produced for the provided instances were checked with the course solution checker, which accepted them as valid (for example, the trivial tour of `gr17` has length 4722).

1 Introduction

The project [1] aims to write a program that reads a TSP instance, calculates a Hamiltonian cycle, and prints it out to a file. It is required that the specification contains a build script located at the root of the project, an executable named `TSPsolver` that reads an instance path, an output path and that may take an optional method name, and an executable named `tests` that reads an instance path and an output path location. It also corrects the input formats: a list of planar coordinates followed by a lower triangular weight matrix, and the output format - a list of permutations terminated with -1 and `EOF`.

The grading puts half of the weight on design quality, so we divided the program into small parts with clear responsibilities. Each of instance representation, distance computation, parsing, validation of tours and output is implemented in a separate class and each has tests.

This report covers the mid-term checkpoint, for which the program must support the trivial method only. For the final submission, in addition to the same parser, graph and output classes, the Held-Karp algorithm will be included, as well as a third algorithm of our own choice.

2 Team and organisation

We split the work by component (Table 1) and agreed early on the shared interfaces, mainly the instance and parser classes, so that each of us could work without waiting for the other. Each member worked on his own feature branch, `riyyan-feature` and `salah_feature`, created from `master`, and we discussed the design together whenever a change affected both parts. Before the checkpoint, `salah_feature` was merged into `riyyan-feature`, which is the branch we submit.

Member	Main responsibilities	Git branch
Riyyan Ahmed	Decision parser (header reading and choice of the data section), `TspInstance`, lower diagonal parser, trivial method, integration for the checkpoint	`riyyan-feature`
Salaheddine Moustafid	Command line (`main`), coordinate parser and `CoordinatesGraph`, build script, test suite, first draft of the report	`salah_feature`

Table 1. Division of the work.

3 Development process

3.1 Timeline

Table 2 lists the main steps of the project. Each push to the repository is given with its commit identifier.

Date	Who	Work	Commit
21 Sep	Both	Reading of the subject and of the TSPLIB format, study of the 15 instance files, instances generated with `instance_generator.py`, background reading on the TSP	–
23 Sep	Both	Division of the work (Table 1)	–
27 Sep	Riyyan	First data structure: distance matrix stored as a `vector<vector<double>>` inside `TspInstance`	–
30 Sep	Riyyan	First push from the local project to the remote repository: instances, subject and CMake template	`877cfb6`, `48d86db`
1 Oct	Riyyan	Decision parser, header reading and lower diagonal parser, empty files for the planned structure	`30e5ed7`
2–9 Oct	Riyyan	Review of the first draft and redesign into graph classes (Section 3.6)	–
10 Oct	Riyyan	Redesigned version: `Graph`, `MatrixGraph`, `TspInstance`, `TspParser`, `TrivialSolver`	`e8f5266`
11 Oct	Salah	Ignore file, `.DS_Store` files removed from version control	`2fa466f`
12 Oct	Salah	Command line and parse, solve, write pipeline in `main`, build script restored	`a43ce1c`, `d9129fc`
12 Oct	Salah	`CoordinatesGraph`, Euclidean distance and coordinate parser	`c9edac5`, `33d98a7`, `b280887`
14–15 Oct	Salah	Review against the specification, bug fixes, test suite (84 tests), first draft of the report	`86d5ae0`
16 Oct	Riyyan	Checkpoint fixes: CMake, build script, command line, city ids starting at 0, README, documentation comments, 92 tests	`4287595`
16 Oct	Riyyan	Editor settings removed from version control	`14ceb99`
17 Oct	Riyyan	Tours of the 15 instances, validation with the course checker, figures	`052319b`
17 Oct	Both	Report completed (written alongside the code throughout the project)	–

Table 2. Timeline of the project. Commit identifiers link to the repository.

3.2 Setting up the project

We split up the work on 23 September so we began implementing it locally, and on 30 September, we pushed our first work to the remote repository. This project is based on a template provided in the course, which has already separated headers, sources and tests. This layout was retained. Together with the code for the lower triangular matrix format, we wrote the first version of the parser on 1 October together with the empty files for the rest of the architecture to fix the structure before the algorithm existed.

The header of the TSPLIB had to be taken care of a little bit. The order of the entries is arbitrary, there may be no space in the files before the separator, some entries (e.g. comments) may include another separator within the value, and there may be any number of files. Thus, each line of the header is split only at the first separator, both the left and right parts of the split line are trimmed, and entries that are not required are discarded.

3.3 Matrix instances and the first solver

By 10 October the parser was able to deal with the lower triangular format and the trivial method existed. Most of the work in this phase was due to two properties of the provided files. First, the matrix may not be written row-wise, that is, one row per line, so it's read as a stream of integers and reconstructed based on the dimension. Second, one instance (`dantzig42`) is followed by a display data section containing coordinates which are only intended for display. This section needs to be recognised explicitly since the parser otherwise rejects data after the matrix. It delays to the next key word, and ignores any following information.

3.4 Command line and coordinate instances

On 12 October we wrote the command line handling and connected the parser, the solver and the writer in `main`, so that the complete pipeline ran for the trivial method on the matrix instances. We lost the build script for a few minutes during a cleanup, and restored that same night to see how easy it can be to mess up the single requirement that is checked first. We then added the coordinate format in three small steps, the graph class, the Euclidean distance and the parsing of the coordinate section.

Since nothing forces the lines of a coordinate file to be sorted, each city is stored at the position given by its identifier, and the identifiers are kept so that the tour file uses those of the input (Section 5.4).

3.5 Review and tests

We reviewed code with the specification and wrote the test suite (on 14 October). Our manual runs did not detect some of the defects the review found: the test files were ignored by Git, an out of range matrix index caused undefined behaviour, the build did not yield `build/TSPsolver` (challenges 3 and 4 in Table 3) and the coordinate parser accepted data after `EOF`. The most informative one had to do with the tour writer. It opened the output file, which will delete the file's contents when reading that file, before verifying that the tour is valid, so a bad solver would have deleted a previous result. The check is now performed before opening a file, and a test ensures that if an invalid tour is entered, no changes are made to an existing file.

The choices we make for these points are discussed and the alternatives left in the language

#	Challenge	Cause	Solution	Commit
1	First data structure did not hold up	One raw matrix inside `TspInstance`, circular include with `TspParser.hpp`, private fields only reachable by the parser, coordinates lost	Class hierarchy based on the graph lab: abstract `Graph`, `MatrixGraph`, `CoordinatesGraph`; `TspInstance` owns its graph and has getters	`30e5ed7` → `e8f5266`
2	Irregular instance files	`KEY: v` and `KEY : v`, values containing `:`, matrix wrapped over lines, `DISPLAY_DATA_SECTION` in `dantzig42`, no `EOF` in `pr1002`	Split on the first `:` and trim, weights read as a stream, display section skipped, `EOF` optional but data after it rejected	`30e5ed7`, `e8f5266`, `86d5ae0`
3	Build not matching the specification	Executable named `main` in the wrong folder, tests not built, global `-Werror` breaking GoogleTest	Warnings as errors only on our targets, executable renamed `TSPsolver`, build into `./build` with the tests	`e8f5266`, `86d5ae0`
4	Bugs found only by the tests	`tests/` ignored by Git, out of range matrix index (undefined behaviour), invalid tour emptied the output file	Ignore rule removed, index checks with exceptions, tour validated before the file is opened, regression tests	`86d5ae0`
5	Generated instances rejected	`instance_generator.py` numbers cities from 0, the parser required ids from 1	Ids may start at 0 or 1, cities stored by id, tests for both cases	`4287595`

Table 3. Main challenges, their cause, our solution and the commit where it was fixed.

by the subject are discussed in Section 12.

3.6 Main challenges

Table 3 summarises the five main challenges we met, with their cause, our solution and the commit in which each was fixed.

The first and third of Table 3 were the two most difficult challenges. After the first push, we had to replace our first data structure, and in the process, we had to rewrite the instance class and part of the parser. But it was worth itbecause we later added the coordinate format using 3 small commits without changing the parser. The build issues were more subtle, since the program ran on our machines, but the exact commands of the specification did not; since then we run build from a clean checkout of the repository, as a grader would do.

4 Tools and libraries

The solver is written in C++20 and uses only the standard library. The data structures needed here (vectors, strings, streams and smart pointers) are all available there, and avoiding other dependencies keeps the build simple on the machines used for grading. The project is built with CMake, and our own targets are compiled with warnings treated as errors, which keeps the code free of unused variables and sign comparison problems.

The tests use GoogleTest [5], which CMake downloads when the project is configured. It is the only external component, and only the test executable uses it. We chose it because it supports parameterised tests, which let us run the same checks on every provided instance.

Table 4 lists all the tools we used.

Tool	Used for
C++20 and its standard library	All the code of the solver, without other dependencies
CMake	Building the library, `TSPsolver` and `tests`
GoogleTest	Unit tests and tests of the executable (downloaded by CMake)
Git and GitHub	Version control, one feature branch per member
Visual Studio Code on macOS	Writing and debugging the code
`instance_generator.py`	Random instances of a chosen size, beyond the 15 provided files (provided by the course)
Course solution checker	Validation of our tour files, as a Moodle VPL activity and as the script `tsp_check.py` (university network or VPN)
Python 3	Running the generator, the checker script and a small timing script
LaTeX	Writing and compiling this report

Table 4. Tools used in the project.

5 Design and code structure

5.1 Overview

The code is structured as a static library with all the logic in it, and two executables that rely on it. The first executable file, `TSPsolver`, has only one function: the entry point. The second, `tests`, uses the same library. This separation is important for testing because no test should be required to call the program through its entry point to get to the parser or to the solvers. Figure 1 shows the directories of the project.

```
TSP-Solver-Project/
|-- build-TSPsolver.sh       build script (creates build/TSPsolver, build/tests)
|-- README.md
|-- TSP-instances/           provided instances: coord/ and lower_diag/
|-- instance_generator.py    random instance generator (course)
|-- tsp_check.py             solution checker client (course)
|-- docs/                    subject and TSPLIB format
|-- reports/                 this report and its figures
`-- tsp-solver/
    |-- CMakeLists.txt       library, TSPsolver and tests targets
    |-- include/             Graph, MatrixGraph, CoordinatesGraph, TspInstance,
    |                        TspParser, Algorithms (Solver), TourWriter
    |-- src/                 one .cpp per class, the two parsers, main.cpp
    `-- tests/               GoogleTest suites (6 files)
```

Figure 1. Directory layout of the project.

5.2 From a matrix to a graph hierarchy

The first version (commit `30e5ed7`) kept all the instances as one distance matrix in the object `TspInstance`. It was easy to write, but limited the future algorithms to one kind of storage, lost coordinates of coordinate files and had no natural site for the distance computation. For this reason we opted for a small class hierarchy (commit `e8f5266`) similar to the one used for the graph library of the object oriented programming lab [2]. The correspondence is given in Table 5.

Lab 4 class	Our class	What changed for the TSP
`Graph` (abstract)	`Graph` (abstract)	Interface reduced to `size()` and `weight(i,j)`, since a TSP graph is always complete
`WeightedGraph::path_weight`	`Graph::tourLength`	Closes the cycle with the edge back to the start
`AdjacenceMatrixWeightedGraph`	`MatrixGraph`	Weights read from the lower triangle of the file
`CoordinatesGraph`	`CoordinatesGraph`	Coordinates are real numbers, distance is Euclidean
`CompleteGraph`	(merged into `Graph`)	Every TSP graph is complete, so no separate class and no diamond inheritance

Table 5. Classes of the graph lab and their counterpart in the solver.

Figure 2 shows the resulting classes. The parser creates a `TspInstance`, which owns its graph through a `std::unique_ptr<Graph>`. Solvers and the tour writer only read the instance through its getters and the abstract `Graph` interface, so they do not depend on the input format.

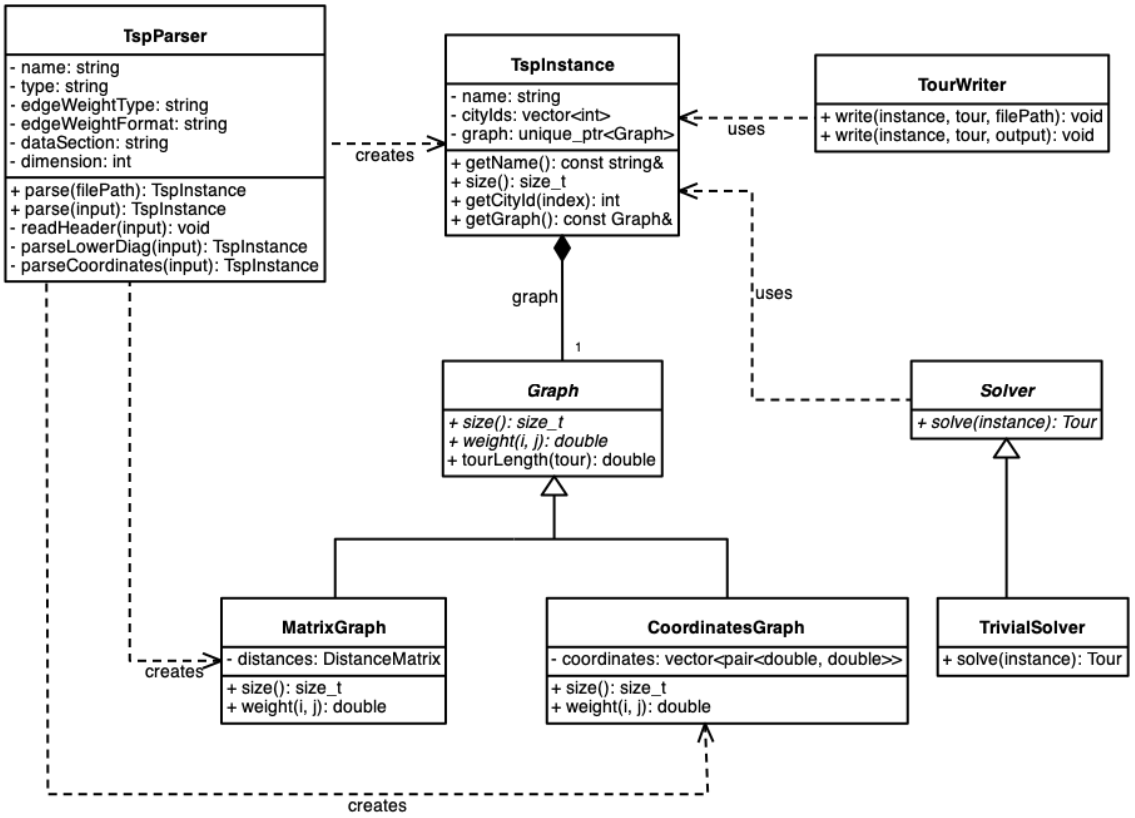


Figure 2. Class diagram of the library.

5.3 Representation of an instance

An instance consists of a name, the identifiers of the cities as given in the file and a graph. The graph is an abstract class and it has two operations one returns the number of vertices, the other returns the weight of a given edge, and a third to calculate the length of a tour. This

function is placed in the base class to ensure that all graph and all solver compute a tour in the same manner.

The lower triangle of this matrix is saved in the matrix graph. The graph is complete, thus an adjacency list would contain as many $n(n-1)/2$ edges, and would require a longer time to find an edge, while the graph would be stored as a dense matrix, and would answer `weight(i,j)` in constant time. The memory requirement of coordinate graph is linear in the number of cities, because it keeps only the points, it calculates all the distances when needed. Both classes have the same interface, so a solver is written once and can be used with both, and both check their inputs and indices and throw exceptions if they are bad, so that a bad input or bad index does not result in undefined behaviour.

In the program, vertices are numbered starting from 0. They're only converted back to the identifiers of the file at the tour writer, taking one of the algorithms with them.

5.4 Parsing

The parser is broken into two phases. The header stage reads the lines of `key : value` till it encounters the keyword at which the data section begins and verifies the presence of the name, the type and the presence of a positive dimension. The keyword then is used to select the data stage, each of which is implemented in a separate file for each format. In addition, the matrix stage needs explicit weights in the matrix format with a row of zeros in the lower diagonal format, because otherwise they will be read incorrectly. The identifier is supplied from 1 to the coordinate stage (for TSPLIB files) or from 0 to the coordinate stage (for course generator); each city is placed on the coordinate stage at the position specified by its identifier.

The parser can be fed anything other than a file path, allowing the tests to provide it with small malformed inputs as strings. One parser can read several instances, each call resets the state of the parser. Errors are reported as exceptions, with the name of the error as a message, and only at the entry point. Exceptions were selected because they are not amenable to local correction when a parsing failure occurs and because they maintain the readability of the normal path of the code.

The parser is strict about content that changes the meaning of the instance (negative weights, a non zero diagonal, duplicate identifiers, missing or trailing data) and tolerant about the rest, such as spacing around separators, blank lines, Windows line endings and unknown header entries.

5.5 Solvers

A solver is any class that implements a common interface with a single function, which takes an instance and returns a tour represented as a vector of vertex indices. The trivial method returns the vertices in increasing order of their index, which is also the order of their identifiers. Because a solver only sees the graph interface and returns a plain vector, it does not depend on the input format or on the output format. Adding a method means adding one class and one line in the function that maps a method name to a solver.

5.6 Writing tours

The tour writer converts a tour to the output format. Before writing, it checks that the tour is a permutation of all vertices, so that no solver can produce an output file that violates

the specification. The output name is derived from the instance name by replacing a trailing extension, and only a trailing one. A second overload writes to any output stream, which is what the tests use to compare the exact text of the output with the example of the specification. Appendix A shows a complete output file.

5.7 Entry point and error handling

The entry point reads the arguments, parses the instance and then calls the solver given in the argument and writes the tour. It requires two arguments: the instance and the output file, and it can take any number of times at the command-line the option `-m` (method name). If any other option is specified, it is a missing or extra path, a `-m` without a value, and a return of non zero status, the method name is checked before reading the instance, and a message is printed with the usage. All errors thrown by the library, both in parsing and in solving or even writing, are exceptions that are derived from the standard exception class. The entry point captures it where it is seen, prints the message from the entry point to the error stream and returns the same status. The library classes do not print anything themselves so they are reusable and the tests check the errors without parsing the text.

Figure 3 shows how these components are connected when the program runs.

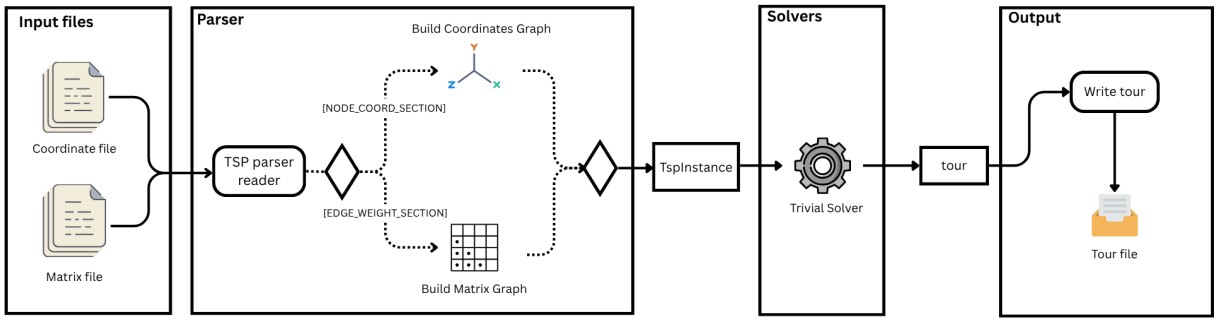


Figure 3. Data flow of a run, from the instance file to the tour file.

6 Key parts of the code

This section shows the short pieces of code that carry the main design decisions. Documentation comments are left out to save space, and some lines are omitted where marked.

The graph interface. Every solver works only with this abstract class, so it is independent of the file format.

```
class Graph {
    public:
        virtual ~Graph() = default;
        virtual std::size_t size() const = 0;
        virtual double weight(std::size_t i, std::size_t j) const = 0;
        double tourLength(const Tour& tour) const;
};
```

Reading the lower triangle. Weights are read one by one, whatever the line breaks, and each value fills both halves of the symmetric matrix.

```
for (int i = 0; i < dimension; i++) {
    for (int j = 0; j <= i; j++) {
        int weight;
        if (!(input >> weight)) {
            throw std::runtime_error("Missing or invalid integer distance");
        }
        // ... checks for negative weights and a non zero diagonal ...
        distances[i][j] = weight;
        distances[j][i] = weight;
    }
}
```

Identifiers starting at 0 or 1. All lines are read first. The smallest identifier tells whether the file counts from 0 (course generator) or from 1 (TSPLIB), and each city is then stored at the position given by its identifier.

```
const int firstId = (*std::min_element(ids.begin(), ids.end()) == 0) ? 0 : 1;

for (int i = 0; i < dimension; ++i) {
    const int index = ids[i] - firstId;
    if (index < 0 || index >= dimension) {
        throw std::runtime_error("City id out of range");
    }
    if (cityIds[index] != -1) {
        throw std::runtime_error("Duplicate city id");
    }
    cityIds[index] = ids[i];
    coordinates[index] = points[i];
}
```

The trivial method. Since the parsers store the cities in increasing order of their identifiers, visiting the indices in order gives the tour $1 \to 2 \to \ldots \to n$.

```
Tour TrivialSolver::solve(const TspInstance& instance) const {
    Tour tour;
    for (std::size_t i = 0; i < instance.size(); i++) {
        tour.push_back(i);
    }
    return tour;
}
```

Writing the tour. The tour is validated before the file is opened, so an invalid tour never empties an existing file.

```
void TourWriter::write(const TspInstance& instance, const Tour& tour,
                       const std::string& filePath) const {
    checkIsPermutation(tour, instance.size());

    std::ofstream output(filePath);
    if (!output.is_open()) {
        throw std::runtime_error("Cannot open output file: " + filePath);
```

```
    }
    write(instance, tour, output);
    // ... check that writing succeeded ...
}
```

7 Complexity

Table 6 gives the time and memory cost of each component, for an instance with n cities.

Component	Time	Memory
Reading the header	$O(h)$, h header lines	$O(1)$
Lower diagonal parser (`MatrixGraph`)	$O(n^2)$	$O(n^2)$, full matrix
Coordinate parser (`CoordinatesGraph`)	$O(n)$	$O(n)$, points only
`weight(i,j)`	$O(1)$	$O(1)$
`tourLength`	$O(n)$	$O(1)$
Trivial method	$O(n)$	$O(n)$
Tour writer, with the permutation check	$O(n)$	$O(n)$
Whole run, matrix file	$O(n^2)$	$O(n^2)$
Whole run, coordinate file	$O(n)$	$O(n)$
Held-Karp (planned)	$O(n^2 2^n)$	$O(n 2^n)$

Table 6. Time and memory complexity of each component.

For the trivial method the cost of a run is dominated by reading the file. For a coordinate file the program never builds a matrix, so memory stays linear in the number of cities.

8 Build and run

The project is built from its root with a single command, which creates `build/TSPsolver` and `build/tests`. GoogleTest is downloaded the first time the project is configured.

```
bash build-TSPsolver.sh
```

The solver takes the instance path, the output path and an optional method. Without `-m`, the trivial method is used.

```
./build/TSPsolver TSP-instances/lower_diag/gr17.tsp gr17.tour
./build/TSPsolver TSP-instances/coord/berlin52.tsp berlin52.tour -m trivial
```

The tests are run with:

```
./build/tests
```

Every invalid use prints a message on the error stream, returns the status 1 and writes no output file. Table 7 lists the main cases.

Situation	Message
No argument, one path, or an extra path	`Error: expected an instance path and an output path`
`-m` without a method name	`Error: -m must be followed by an algorithm name`
Unknown option, e.g. `-x`	`Error: unknown option '-x'.`
Unknown method, e.g. `-m held-karp`	`Error: unknown or not yet implemented algorithm 'held-karp'.`
Instance file not found	`Error: Cannot open file: <path>`
Output folder not found	`Error: Cannot open output file: <path>`
Malformed instance	`Error:` followed by the reason, e.g. `Missing NAME`

Table 7. Error messages of the command line (exit status 1 in every case).

9 Testing and validation

9.1 Unit and executable tests

The test suite contains 92 tests in 11 groups, and it runs in well under a second. We organised it by component so that a failure points to a single class. Table 8 summarises the six test files, and Appendix B lists every test by name.

File	Tests	What is checked
`ParserTests.cpp`	32	Both formats; header variants (spaces around `:`, Windows line endings, comments); missing or invalid name, type and dimension; truncated and surplus data; duplicate and out of range identifiers; identifiers starting at 0; shuffled lines; negative and non integer weights; wrapped matrix; display section; missing or trailing `EOF`; real files
`SolverTests.cpp`	19	Each of the 15 provided instances is parsed and solved (identifiers, zero diagonal, valid permutation, positive length); sizes of some files; symmetry of `gr17`; identity tour; single city
`GraphTests.cpp`	17	Euclidean distance, symmetry, zero diagonal, triangle inequality, invalid input and indices, tour length including the example of the specification (18), closing edge, independence from start and direction
`ExecutableTests.cpp`	13	The real `TSPsolver`: default and explicit method, `-m` before or between the paths, unknown method or option, missing method name, extra or missing arguments, missing instance, unwritable output, output read back as a permutation
`TourWriterTests.cpp`	7	Exact output of the specification example, tour name, identifiers written instead of indices, invalid tours rejected, file writing, existing file kept on error
`InstanceTests.cpp`	4	Getters, missing graph, wrong number of identifiers, out of range identifier
Total	92	

Table 8. Content of the test suite.

Several tests failed before the corrections described in Section 3 and now pass, namely the empty matrix, the invalid matrix index, the protection of existing output files, the data after the final keyword in a coordinate file and the identifiers starting at 0. We therefore consider them regression tests for real defects and not only coverage.

9.2 Instances

Besides the unit tests, we ran the solver on every provided instance and on generated instances (Table 9). The generated instances, created with `instance_generator.py`, number their cities from 0 and range from 1 to 50 000 cities.

Format	Files	Instances
Lower diagonal matrix	5	`gr17`, `gr21`, `gr24`, `dantzig42`, `gr48`
Coordinates	10	`burma14`, `ulysses16`, `att48`, `berlin52`, `st70`, `eil76`, `pr76`, `ch130`, `pr299`, `pr1002`
Generated coordinates	28	1, 5, 20, 100 and 1000 cities (2 files each), and 10, 100, 1000, 5000, 10 000 and 50 000 cities (3 files each)

Table 9. Instances on which the solver was run.

9.3 Validation with the course checker

The tour files of the 15 provided instances were submitted to the course solution checker on Moodle, together with their instance file, and all of them were accepted. For example, for `gr17` the checker reads the instance as an explicit lower triangular matrix of dimension 17, reads our tour $1 \to 2 \to \ldots \to 17 \to 1$ and reports a length of 4722, which is the same value as our own computation. The same check can be run from the command line with the script `tsp_check.py`, from the university network or through its VPN.

10 Performance evaluation

The only method here is trivial, so we use the trivial method as a baseline of the methods in the last checkpoint. All times reported are averages of 5 complete runs (start, read, solve, write) on an Apple M5 laptop running 24 GB of memory and compiled by Apple clang 21 with `-O3` release compilation mode.

Solution quality. Table 10 shows a comparison of the length of the trivial tour with the optimal tour length of each instance. Our program does not compute the optimum - it is the value that is published with TSPLIB [3] for each instance. The trivial lengths are calculated using the distance functions in the TSPLIB package, which is what the course checker does (these give 4722 for `gr17`). The gap is (trivial − optimum)/optimum.

Instance	Trivial	Optimum*	Gap	Instance	Trivial	Optimum*	Gap
`burma14`	4 562	3 323	37 %	`berlin52`	22 205	7 542	194 %
`ulysses16`	9 665	6 859	41 %	`st70`	3 410	675	405 %
`gr17`	4 722	2 085	126 %	`eil76`	1 969	538	266 %
`gr21`	6 620	2 707	145 %	`pr76`	150 781	108 159	39 %
`gr24`	3 436	1 272	170 %	`ch130`	47 797	6 110	682 %
`dantzig42`	699	699	0 %	`pr299`	83 506	48 191	73 %
`att48`	49 840	10 628	369 %	`pr1002`	349 403	259 045	35 %
`gr48`	19 837	5 046	293 %				

Table 10. Length of the trivial tour compared with the optimal tour length. *Published with TSPLIB, not computed by our program.

The trivial tour is 35 % to 682 % longer than the optimum, depending only on how the cities happen to be numbered in each file. The one exception, `dantzig42`, lists its cities in the order of an optimal tour. These values are the reference that the next methods must improve on, and the Held-Karp algorithm will have to reach the published optimum exactly on the small instances.

Time and scalability. The 15 provided instances all run in 1.8 to 8.7 ms (`pr1002` is the slowest). Table 11 shows generated instances of increasing size, three per size.

Cities n	10	100	1 000	5 000	10 000	50 000
Time (ms)	2.1	1.9	2.9	4.3	10.7	17.1

Table 11. Average time of a complete run on generated instances.

Up to about a thousand cities the time is dominated by a fixed cost of about 2 ms for starting the program, so it hardly depends on n. Beyond that it grows with n, as expected from the linear cost of reading and writing the cities (Section 7). At this scale the measurements are noisy, so they give orders of magnitude only. The cost of the solving step itself will matter with the Held-Karp algorithm, whose time grows exponentially.

11 Specification compliance

Table 12 lists the requirements of the subject [1] and where they are met.

Spec.	Requirement	Status	Where
2.1.1	`bash build-TSPsolver.sh` at the root builds the project	Done	Section 8
2.1.1	`build/TSPsolver` and `build/tests` are created	Done	Section 8
2.1.2	`TSPsolver <instance> <output>` runs the default method	Done	Section 8
2.1.3	`-m trivial`	Done	Section 8
2.1.3	`-m held-karp` and a third method	Final submission	Section 13
2.1.4	`./build/tests` runs all the tests	Done (92 tests)	Section 9
2.2	Coordinate files (`NODE_COORD_SECTION`)	Done (10 files)	Table 9
2.2	Lower diagonal files (`LOWER_DIAG_ROW`)	Done (5 files)	Table 9
2.3	Tour file format	Done, accepted by the checker	Section 9
2.4.1	Trivial method	Done	Section 6
2.4.2	Held-Karp algorithm	Final submission	Section 13
2.5	Performance evaluation	Baseline done	Section 10
3	Mid-term version supporting the trivial method	Done	This report
3.2	Report as a PDF document	Done	This report

Table 12. Requirements of the subject and their status at the checkpoint.

12 Limitations and decisions

The subject leaves some points open. Table 13 lists the decisions we took.

The major constraint is the final decision. The distance functions defined in TSPLIB are: rounded Euclidean (`EUC_2D`), geographical (`GEO`) and pseudo Euclidean (`ATT`). For all of our files, our coordinate graph is designed with a plain Euclidean distance but not rounded distances.

Point	Decision
`EOF` keyword	Optional, since `pr1002` has none; any data after it is rejected
`DISPLAY_DATA_SECTION`	Skipped, since it only holds drawing coordinates (`dantzig42`)
City identifiers	May start at 1 (TSPLIB) or at 0 (course generator); the tour file uses the identifiers of the input
`NAME` of the tour file	Instance name with a trailing `.tsp` removed, followed by `.tour`
Default method	Trivial
Command line	Exactly two paths, `-m` anywhere, any other argument rejected
Distances of coordinate files	Plain Euclidean distance, whatever `EDGE_WEIGHT_TYPE` says

Table 13. Decisions on points left open by the subject.

The tours written by us are not affected, as it is a trivial method that isn't based on distance, but the lengths of the tours that our program produces would differ from the official ones. The difference is small for `EUC_2D` files (`eil76`: 1974.7 vs. 1969), but large for the other types of file because `GEO` coordinates are latitudes/longitudes and `ATT` uses a scaled distance (`burna14`: 42.5 vs. 4562; `att48`: 157 530 vs. 49 840). This should be corrected before comparing the lengths of the following methods with published optima.

The second drawback is that this is the trivial solution and does not attempt to find a short tour, as it is designed to test the pipeline from parsing to output (Table 10).

13 Future work

For the final submission we plan the following steps, in this order.

1. **TSPLIB distances.** Read `EDGE_WEIGHT_TYPE` and give the coordinate graph the matching distance function (`EUC_2D`, `GEO`, `ATT`), and check the lengths against the course checker.

2. **Held-Karp algorithm** [4]. Dynamic programming over subsets with a bitmask, plus backtracking to recover the tour. Its table needs $n \cdot 2^{n-1}$ values, about 84 MB for $n = 20$ and 3.4 GB for $n = 25$, so the solver will refuse instances above a size limit instead of running out of memory. It must give the known optima, for example 2085 on `gr17` and 3323 on `burma14`.

3. **Third method.** A heuristic for large instances, for example a nearest neighbour tour improved by 2-opt, to be discussed with the teacher before coding.

4. **Measurements.** Time the solving step alone inside the program, print the tour length, and repeat Section 10 for the three methods.

14 Reflections

The most useful decision of the first weeks was to fix the structure of the code early. It allowed us to add the second input format in three small commits without touching existing code. The review of 14 October was less comfortable, since it showed that several parts we considered complete had behaviour that we had never exercised. The problems with the output file and the matrix accessor would not have appeared in normal use, and we only found them because the tests supplied inputs that a careful user would not. If we repeated the project, we would write the tests of each component at the same time as the component.

Two further lessons came from the last days before the checkpoint. First, a program that works on our machines is not enough: the build has to be checked exactly as the grader will run it, from a clean copy and with the commands of the subject, which is how we found that the executable had the wrong name and location. Second, working on two separate branches for two weeks made the final merge harder than it needed to be. Agreeing on the shared classes first helped, but merging more often would have shown integration problems earlier.

15 Use of AI tools

Tools used

- Claude Opus 5.5 and Claude Sonnet 5.5 (Anthropic), between 25 September and 13 October 2026.

AI-generated code

Each part is marked in the code with a comment naming the tool and the date.

- `src/TspParser.cpp`, function `trim()` (Claude Opus 5.5, 25 September): the helper that removes spaces, tabs and line breaks at both ends of a header line.
- `src/MatrixGraph.cpp`, constructor checks (Claude Opus 5.5, 7 October): the two checks that reject an empty or a non-square distance matrix.
- `tests/ParserTests.cpp` (Claude Sonnet 5.5, 13 October): formatting of the small instance texts used as test input, and some of the parser tests.
- `tests/ExecutableTests.cpp` (Claude Sonnet 5.5, 13 October): the helper `slurp()`, which reads a whole output file into a string, and part of the `TEST_F` tests that run the executable.

Code written without AI

- All other source files, headers and tests, the CMake file and the build script.

How we checked the AI-generated parts

- We read and understood each part before keeping it, and adapted it to our code.
- Every part is covered by the test suite (92 tests passing), and the output tours were validated with the course checker.

Other uses of AI (not code)

- Explanations of C++ concepts while learning (e.g. `std::move`, `unique_ptr`, virtual functions, parsing with streams and other complex concepts).
- Review of the code against the specification, to find missing requirements and bugs.

DALIA self-assessment

- **D2, AI-Augmented** (daliascale.org): generative AI contributed without altering the meaning of the work; human intent, direction and judgment dominated throughout.

References

[1] E. Foussard. *Project – Solving the Travelling Salesman Problem*. Programming course, M1 AI, Université Grenoble Alpes, 2026.

[2] E. Foussard. *TP4 – Object-Oriented Programming and Graphs*. Programming course, M1 AI, Université Grenoble Alpes, 2026.

[3] G. Reinelt. *TSPLIB 95*. Universität Heidelberg, 1995. Optimal tour lengths, archived copy: `https://web.archive.org/web/20231203135812/http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/STSP.html`

[4] M. Held and R. M. Karp. A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1):196–210, 1962.

[5] Google. *GoogleTest*, version 1.17.0. `https://github.com/google/googletest`

A Example of an output file

Tour file written by `./build/TSPsolver TSP-instances/lower_diag/gr17.tsp gr17.tour`. The lines for cities 4 to 14 are left out.

```
NAME : gr17.tour
TYPE : TOUR
DIMENSION : 17
TOUR_SECTION
1
2
3
...
15
16
17
-1
EOF
```

B List of tests

The 92 tests of `./build/tests`, grouped by file. `ParseAndTrivialSolve` runs once for each of the 15 provided instances.

ParserTests.cpp (32)
`ReadsCoordinateInstance`, `ReadsLowerDiagInstance`, `AcceptsSpaceBeforeColonAndWindowsLineEndings`, `IgnoresCommentsAndUnknownHeaderEntries`, `AcceptsMissingFinalEof`, `AcceptsScientificNotationAndNegativeCoordinates`, `StoresCoordinatesByIdWhenLinesAreShuffled`, `SkipsDisplayDataSectionAfterMatrix`, `ParserCanBeReusedAcrossInstances`, `RejectsMissingFile`, `RejectsEmptyInput`, `RejectsMissingName`, `RejectsWrongType`, `RejectsMissingOrInvalidDimension`, `RejectsHeaderLineWithoutColon`, `RejectsMissingDataSection`, `RejectsTruncatedCoordinates`, `RejectsNonNumericCoordinate`, `RejectsOutOfRangeAndDuplicateCityIds`, `AcceptsIdsStartingAtZeroLikeTheInstanceGenerator`, `StoresZeroBasedIdsByIdWhenLinesAreShuffled`, `RejectsExtraCoordinateLines`, `RejectsMatrixWithWrongFormat`, `RejectsInvalidMatrixValues`, `RejectsTrailingDataAfterMatrix`, `AcceptsMatrixWithoutFinalEof`, `AcceptsMatrixWrappedOverArbitraryLines`, `IgnoresBlankLinesAndIndentationInHeaderAndData`, `RejectsUnexpectedDataAfterCoordinates`, `RejectsMatrixShorterThanDimension`, `RejectsNonIntegerMatrixWeight`, `ParsesEveryProvidedInstanceWithItsDeclaredDimension`

SolverTests.cpp (19)
`ParseAndTrivialSolve` (15 instances), `MatchTheirNames`, `Gr17MatrixIsSymmetric`, `ReturnsIdentityPermutation`, `HandlesSingleCity`

GraphTests.cpp (17)
`ComputesEuclideanDistance`, `IsSymmetricWithZeroDiagonal`, `SatisfiesTriangleInequality`, `RejectsEmptyInput`, `RejectsOutOfRangeIndex` (coordinates), `ReturnsStoredWeights`, `RejectsNonSquareMatrix`, `RejectsEmptyMatrix`, `RejectsOutOfRangeIndex` (matrix), `MatchesSpecificationExample`, `ClosesTheCycle`, `IsIndependentOfStartAndDirection`, `SingleCityAndEmptyTourHaveZeroLength`, `RejectsInvalidVertex`, `ChecksEveryVertexNotOnlyTheFirstOne`, `PrefersPerimeterOrderOnASquare`, `CoincidentCitiesAreAtDistanceZero`

ExecutableTests.cpp (13)
`SolvesWithDefaultMethod`, `AcceptsExplicitTrivialMethod`, `FailsOnUnknownAlgorithmWithoutWritingOutput`, `FailsOnMissingInstance`, `FailsWithoutArguments`, `AcceptsMethodBeforeThePaths`, `AcceptsMethodBetweenThePaths`, `FailsWhenMethodNameIsMissing`, `FailsOnUnknownOption`, `FailsOnExtraArgument`, `FailsWithOnlyOneArgument`, `FailsOnUnwritableOutput`, `OutputCanBeReadBackAsAPermutation`

TourWriterTests.cpp (7)
`WritesSpecificationFormat`, `OnlyReplacesTspExtensionAtTheEnd`, `OutputsCityIdsNotIndices`, `RejectsInvalidTours`, `WritesFile`, `FailsOnUnwritablePath`, `InvalidTourDoesNotTouchExistingFile`

InstanceTests.cpp (4)
`ExposesNameSizeAndIds`, `RejectsMissingGraph`, `RejectsIdCountMismatch`, `RejectsOutOfRangeCityId`